

```

import TelegramBot from 'node-telegram-bot-api';
import { CONFIG } from './config.js';
import { getJobById, updateJobStatus } from './db.js';
import { applyToJob } from './applicator.js';

let bot;

export function initTelegram() {
  bot = new TelegramBot(CONFIG.telegram.token, { polling: true });

  bot.on('callback_query', async (query) => {
    const [action, jobId] = query.data.split(':');
    const job = getJobById(jobId);
    if (!job) {
      bot.answerCallbackQuery(query.id, { text: 'Vaga não encontrada' });
      return;
    }

    if (action === 'apply') {
      bot.answerCallbackQuery(query.id, { text: '🕒 Aplicando...' });
      await bot.editMessageText(
        formatJobMessage(job) + '\n\n🕒 *Aplicando agora...*',
        {
          chat_id: query.message.chat.id,
          message_id: query.message.message_id,
          parse_mode: 'Markdown',
        }
      ).catch(() => {});

      try {
        await applyToJob(job);
        updateJobStatus(jobId, 'applied');
        await bot.editMessageText(
          formatJobMessage(job) + '\n\n✅ *Aplicado com sucesso*',
          {
            chat_id: query.message.chat.id,
            message_id: query.message.message_id,
            parse_mode: 'Markdown',
          }
        );
      } catch (err) {
        updateJobStatus(jobId, 'failed');
        await bot.editMessageText(
          formatJobMessage(job) + '\n\n❌ *Falha: * ${err.message}`,

```

```

        {
            chat_id: query.message.chat.id,
            message_id: query.message.message_id,
            parse_mode: 'Markdown',
        }
    );
}
} else if (action === 'skip') {
    updateJobStatus(jobId, 'skipped');
    bot.answerCallbackQuery(query.id, { text: '🔊 Pulado' });
    await bot.editMessageText(
        formatJobMessage(job) + '\n\n🔊 *Pulado*',
        {
            chat_id: query.message.chat.id,
            message_id: query.message.message_id,
            parse_mode: 'Markdown',
        }
    );
} else if (action === 'edit') {
    bot.answerCallbackQuery(query.id);
    await bot.sendMessage(
        query.message.chat.id,
        `Cover letter atual:\n\n\`\`\`\n${job.cover_letter}\n\`\`\`\n\nResponda e
        { parse_mode: 'Markdown', reply_markup: { force_reply: true } }
    );
}
});

bot.on('message', async (msg) => {
    // Se é reply de "edit cover letter", pega o novo texto
    if (msg.reply_to_message?.text?.includes('Responda essa mensagem com o novo t
        // (extensão futura: atualizar cover letter e reaplicar)
    }
});

return bot;
}

function formatJobMessage(job) {
    const budget = job.budget_type === 'hourly'
        ? `💰 ${job.budget_min || '?'}-${job.budget_max || '?'}/hr`
        : `💰 ${job.budget_min || '?'}-${job.budget_max || '?'} fixed`;

    return `🎯 *${escapeMd(job.title)}*

    ${budget} | 🇧🇷 Match: *${job.match_score}/10* | 🏠 ${job.category}

```

 Por quê: `${escapeMd(job.match_reasoning)}`

✍️ *Cover letter:*

\\ \\ \\ \\

```
${job.cover_letter}
```

\\'\\'\\'

```
Ⓛ [Ver vaga] (${job.url}) `;
```

}

```
function escapeMd(text) {
  return (text || '').replace(/([_*(\[ \] () ~`>#+=|{} .!-])/g, '\\$1');
}
```

```
export async function notifyJob(job) {
  const keyboard = {
    inline_keyboard: [[
      { text: '✅ Aplicar', callback_data: `apply:${job.id}` },
      { text: '✎ Editar', callback_data: `edit:${job.id}` },
      { text: '⏮ Pular', callback_data: `skip:${job.id}` },
    ]],
  };

  const msg = await bot.sendMessage(
    CONFIG.telegram.chatId,
    formatJobMessage(job),
    { parse_mode: 'Markdown', reply_markup: keyboard, disable_web_page_preview: f
  );

  return msg.message_id;
}
```

```
export async function notifyText(text) {
  if (!bot) initTelegram();
  return bot.sendMessage(CONFIG.telegram.chatId, text, { parse_mode: 'Markdown' })
}
```